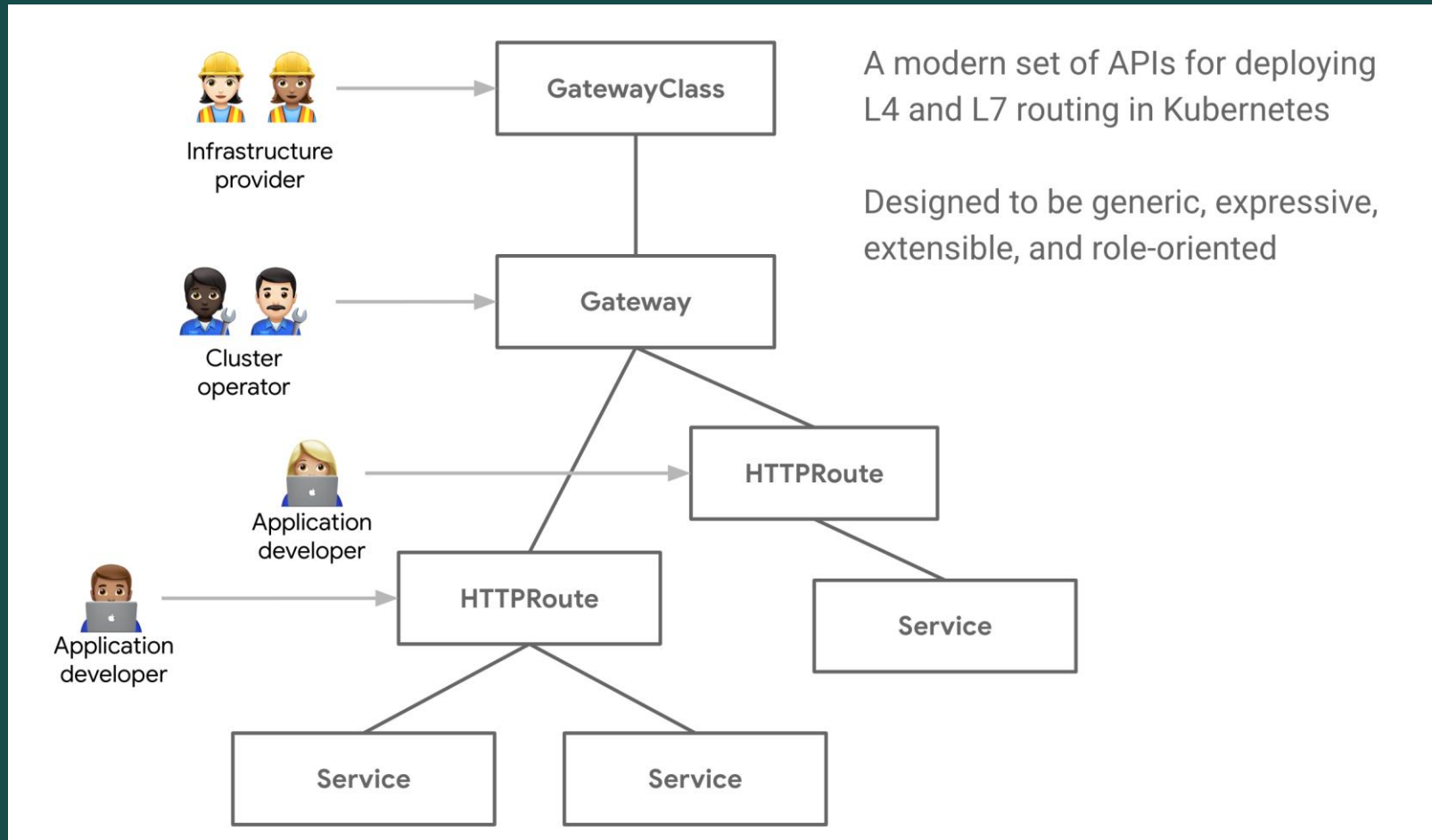


Gateway API basics

- **Gateway API** is a new **service networking model** in Kubernetes that aims to provide a more **expressive**, **extensible**, and **role-oriented** interface for managing network gateways and routing traffic
- **Open-source** project
- Gateway API is a **successor to the Ingress API** and **supports more protocols and features**, such as HTTP, TLS, TCP, UDP, gRPC, redirects, rewrites, traffic splitting, cross-namespace routing, and TLS configuration

Gateway API resources

- **GatewayClass**: Defines gateway type, config options. Equivalent to IngressClass
- **Gateway**: Routes traffic to Services. Configurable with listeners, routes, TLS options. References GatewayClasses for implementation details
- **Route**: Sets rules for traffic routing to Services. Supports HTTP, TCP, TLS, UDP, gRPC. Attaches to Gateways for filtering and modification



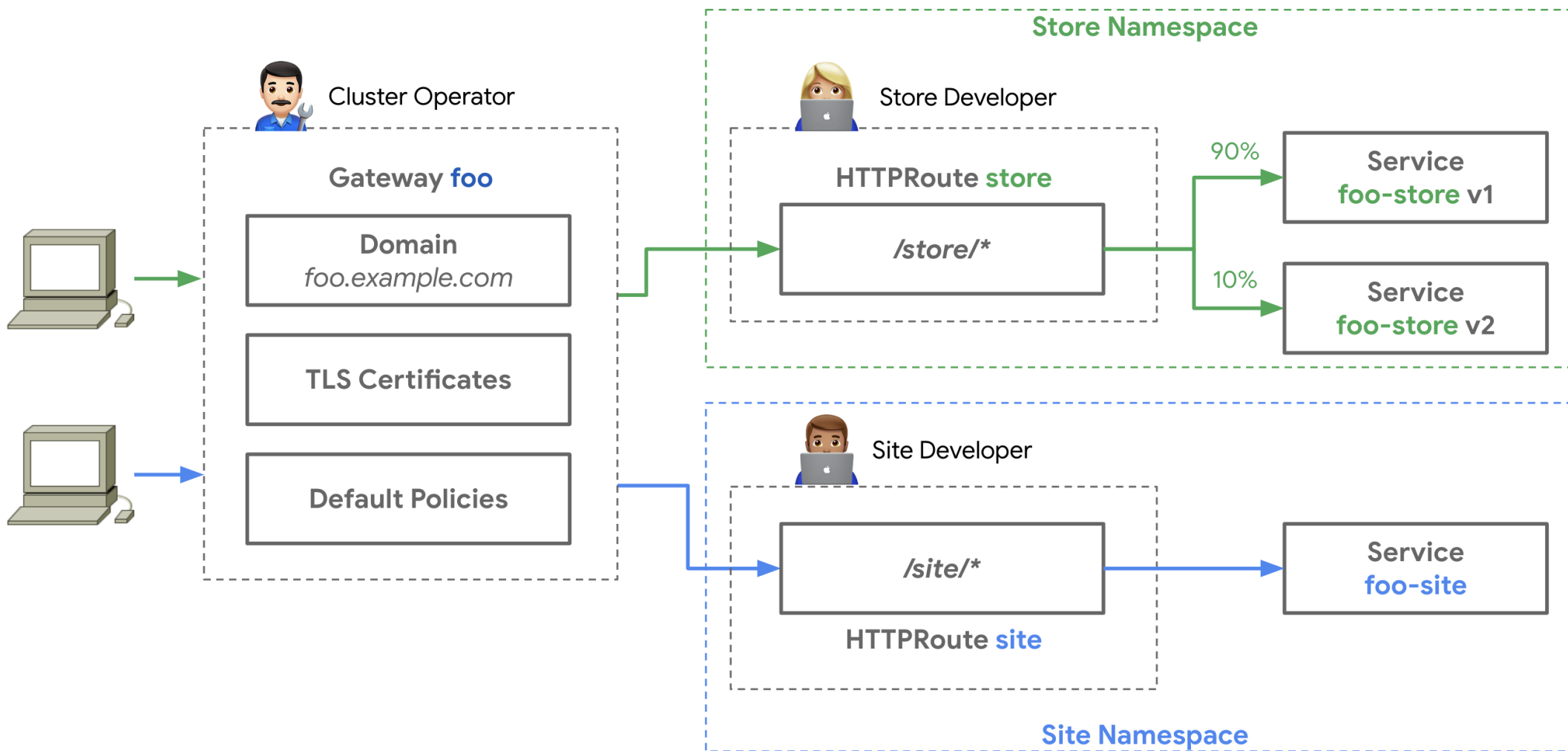
Source: <https://gateway-api.sigs.k8s.io/>

Gateway API basics

Why you might want to switch from the Ingress API to the Gateway API?

- **Ingress poses risks:** Shared resource causes conflicts, errors, security issues for both users and owners.
 - Gateway API for balance: **Offers multiple resources, delegation to different roles** (GatewayClass, Gateway, HTTPRoute, TCPRoute, etc.)
- **Ingress lacks flexibility:** Configuration mainly at the ingress level, leading to sharing conflicts among users and security concerns
 - Gateway API's multiple resources **facilitate Kubernetes RBAC**, enabling fine-grained authorization for specific users, ensuring access privileges are precisely controlled
- **Ingress has a few configurable fields and relies on annotations** for additional features
 - Gateway API is **more portable and interoperable** across different implementations and environments
- **Ingress does not support advanced routing scenarios**, such as traffic splitting, header-based routing, or path rewriting
 - **Gateway API supports a wider range of features and capabilities**, such as protocol negotiation, request/response modification, retries and timeouts, custom filters and extensions, and more

Gateway API basics



Gateway API basics

Feature	Ingress API	Gateway API
Stability	Stable since Kubernetes 1.19¹	Still in alpha stage²
Scope	Cluster-wide¹	Namespace-scoped²
Customizability	Limited by annotations¹	Richer by custom resources²
Extensibility	Depends on the ingress controller¹	Supports multiple protocols and backends²
Separation of concerns	Cluster operator and application developer share the same ingress object³	Cluster operator and application developer use different gateway objects³
Implementations	Many ingress controllers available, such as nginx, traefik, istio, etc.⁴	Few gateway controllers available, such as contour, istio, kong, etc.⁵
Routing Feature	Ingress API	Gateway API
Multiple hosts	Supported by using multiple rules with different host names¹	Supported by using multiple listeners with different host names²
Path based	Supported by using path and pathType fields¹	Supported by using path and pathType fields²
Query string	Not supported natively, requires custom annotations³	Supported by using matchExpressions field²
Header	Not supported natively, requires custom annotations³	Supported by using matchExpressions field²
Traffic splitting	Not supported natively, requires custom annotations³	Supported by using weight field²
Redirects	Not supported natively, requires custom annotations⁴	Supported by using filters field²
Rewrites	Not supported natively, requires custom annotations⁴⁵	Supported by using filters field²
Timeouts	Not supported natively, requires custom annotations⁶	Supported by using timeoutPolicy field²
Retries	Not supported natively, requires custom annotations⁶	Supported by using retryPolicy field²
Health checks	Not supported natively, requires custom annotations	Supported by using healthCheckPolicy field²